

LTE Standard(U)系列 QuecOpen(SDK)

SPI NOR Flash API 参考手册

LTE Standard 模块系列

版本：1.1

日期：2023-09-14

状态：受控文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司
上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233
电话：+86 21 5108 6236 邮箱：info@quectel.com

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：
<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：support@quectel.com。

前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

使用和披露限制

许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他软硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2023，保留一切权利。

Copyright © Quectel Wireless Solutions Co., Ltd. 2023.

文档历史

修订记录

版本	日期	作者	变更表述
-	2021-03-02	Ryan YI	文档创建
1.0	2021-08-27	Ryan YI	受控版本
1.1	2023-09-14	Sum LI	- 基于 QuecOpen 方案统一命名，更新文档名称。 - 新增适用模块 EC200G-CN、EC600G 系列、EC800G-CN、EG700G-CN、EG800G 系列、EG912U-GL 和 EG915U 系列。 - 删除有关模块支持的外接 NOR flash 型号的备注（第 1 章）。 - 新增 SPI NOR flash 型号适配（第 2 章）。 - 新增 4 线 SPI NOR flash API 函数（第 3.1.3.3~3.1.3.6 章）： ql_spi_nor_write_8bit_status(); ql_spi_nor_write_16bit_status(); ql_spi_nor_read_status_low(); ql_spi_nor_read_status_high()。 - 更新 4 线 SPI NOR flash 结果码枚举 ql_errcode_spi_nor_e 定义（第 3.1.3.1.1 章）。 - 新增 4 线 SPI NOR flash 文件系统挂载 API 函数（第 3.2 章）： ql_spi4_ext_nor_sffs_set_port(); ql_spi4_ext_nor_sffs_get_port(); ql_spi4_ext_nor_sffs_mount(); ql_spi4_ext_nor_sffs_unmount(); ql_spi4_ext_nor_sffs_mkfs(); ql_spi4_ext_nor_sffs_is_mount()。 - 新增 4 线 SPI NOR flash 文件系统挂载 API 开发示例和功能调试（第 4.1.2、4.2.2 章）。 - 新增有关取消 NOR flash 写保护操作的描述（第 4.2.1 章）。

目录

文档历史.....	3
目录.....	4
表格索引.....	6
图片索引.....	7
1 引言.....	8
1.1. 适用模块.....	8
2 SPI NOR Flash 型号适配.....	10
2.1. 支持的 NOR Flash 型号信息.....	10
2.2. 相关参数含义.....	11
2.2.1. quec_spi_nor_flash_info_s.....	11
2.2.1.1. quec_spi_nor_flash_status_mode_e.....	11
2.2.1.2. 状态寄存器.....	12
3 SPI NOR Flash API.....	14
3.1. 4 线 SPI NOR Flash API.....	14
3.1.1. 头文件.....	14
3.1.2. 函数概览.....	14
3.1.3. 函数详解.....	15
3.1.3.1. ql_spi_nor_init.....	15
3.1.3.1.1. ql_errcode_spi_nor_e.....	15
3.1.3.1.2. ql_spi_port_e.....	17
3.1.3.1.3. ql_spi_clk_e.....	17
3.1.3.1.4. ql_spi_input_sel_e.....	20
3.1.3.1.5. ql_spi_transfer_mode_e.....	20
3.1.3.1.6. ql_spi_cs_sel_e.....	21
3.1.3.2. ql_spi_nor_init_ext.....	21
3.1.3.2.1. ql_spi_nor_config_s.....	22
3.1.3.3. ql_spi_nor_write_8bit_status.....	23
3.1.3.4. ql_spi_nor_write_16bit_status.....	23
3.1.3.5. ql_spi_nor_read_status_low.....	24
3.1.3.6. ql_spi_nor_read_status_high.....	24
3.1.3.7. ql_spi_nor_read.....	25
3.1.3.8. ql_spi_nor_write.....	25
3.1.3.9. ql_spi_nor_erase_chip.....	26
3.1.3.10. ql_spi_nor_erase_sector.....	26
3.1.3.11. ql_spi_nor_erase_64k_block.....	27
3.2. 4 线 SPI NOR Flash 文件系统挂载 API.....	27
3.2.1. 头文件.....	27
3.2.2. 函数概览.....	27
3.2.3. 函数详解.....	28
3.2.3.1. ql_spi4_ext_nor_sffs_set_port.....	28

3.2.3.2.	ql_spi4_ext_nor_sffs_get_port	28
3.2.3.3.	ql_spi4_ext_nor_sffs_mount	29
3.2.3.3.1.	ql_errcode_spi4_extnsffs_e.....	29
3.2.3.4.	ql_spi4_ext_nor_sffs_unmount	30
3.2.3.5.	ql_spi4_ext_nor_sffs_mkfs.....	30
3.2.3.6.	ql_spi4_ext_nor_sffs_is_mount.....	31
3.2.3.6.1.	ql_spi4_extnsffs_status_e	31
4	示例	32
4.1.	开发示例	32
4.1.1.	4 线 SPI NOR Flash API 开发示例	32
4.1.2.	4 线 SPI NOR Flash 文件系统挂载 API 开发示例.....	32
4.2.	功能调试	33
4.2.1.	4 线 SPI NOR Flash API 功能调试	33
4.2.2.	4 线 SPI NOR Flash 文件系统挂载 API 功能调试.....	35
5	附录 参考文档及术语缩写	37

表格索引

表 1: 适用模块	8
表 2: 函数概览	14
表 3: 函数概览	27
表 4: 参考文档	37
表 5: 术语缩写	37

图片索引

图 1: 支持的 NOR Flash 型号相关参数信息	10
图 2: XM25QU64B 状态寄存器	12
图 3: GD25LQ64E 状态寄存器	13
图 4: XM25QU128C 状态寄存器	13
图 5: 入口函数 <code>ql_spi_nor_flash_demo_init()</code>	32
图 6: 4 线 SPI NOR Flash API 示例默认不启动	32
图 7: 入口函数 <code>ql_spi4_ext_nor_sffs_demo_init()</code>	33
图 8: 4 线 SPI NOR Flash 文件系统挂载 API 示例默认不启动	33
图 9: COM 设备图 1 (ECx00U 系列/EGx00U 系列/EG91xU 系列模块)	33
图 10: COM 设备图 2 (ECx00G 系列/EGx00G 系列模块)	34
图 11: Log 信息	34
图 12: 取消 NOR Flash 写保护操作	34
图 13: 4 线 SPI NOR Flash 文件系统挂载 Log 信息	35
图 14: Dump Log 信息	36

1 引言

移远通信 LTE Standard(U)系列模块支持 QuecOpen®方案；QuecOpen®是基于 RTOS 的嵌入式开发平台，可简化 IoT 应用的软件设计和开发过程。有关 QuecOpen®的详细信息，请参考文档 [1]。

本文档适用于 CSDK 构建环境的 QuecOpen®方案，主要介绍 LTE Standard(U)系列模块的外接 4 线 SPI NOR flash 型号适配、4 线 SPI NOR flash API、4 线 SPI NOR flash 文件系统挂载 API 及相关开发示例和功能调试。

1.1. 适用模块

表 1: 适用模块

模块系列	模块
ECx00G	EC200G-CN
	EC600G 系列
	EC800G-CN
ECx00U	EC200U 系列
	EC600U 系列
EGx00U	EG500U-CN
	EG700U-CN
EGx00G	EG700G-CN
	EG800G 系列
EG91xU	EG912U-GL
	EG915U 系列

备注

本文档不适用于 EC200D 系列模块。

2 SPI NOR Flash 型号适配

模块 QuecOpen CSDK 代码中提供了 NOR flash 配置文件 *quec_spi_nor_flash_prop.c* 和 *quec_spi_nor_flash_prop.h*，位于 `\components\ql_kernel\drivers\` 目录下。用户可在配置文件中自行添加相应的 NOR flash 型号信息。

2.1. 支持的 NOR Flash 型号信息

模块支持的部分外接 NOR flash 型号信息已在 *quec_spi_nor_flash_prop.c* 中的 *quec_spi_nor_flash_props* 内定义，例如：

```

/* 4-line SPI NOR Flash supported model list, can add by yourself */
static quec_spi_nor_flash_info_s quec_spi_nor_flash_props[] = {
{
    // You can cut out it by defining macros
    // #define DISABLE_GD25LE64E_C
    #ifndef DISABLE_GD25LE64E_C
    {
        // GD25LE64E
        // GD25LE64C
        .mid = 0x1760c8,
        .capacity = 0x800000,
        .mode = QUEC_NOR_FLASH_MODE_16BIT,
        .mount_start_addr = 0x0000000,
        .mount_size = 0x800000,
    },
    #endif
    #ifndef DISABLE_XM25QU64B
    {
        // XM25QU64B
        .mid = 0x175020,
        .capacity = 0x800000,
        .mode = QUEC_NOR_FLASH_MODE_8BIT,
        .mount_start_addr = 0x0000000,
        .mount_size = 0x800000,
    },
    #endif
    #ifndef DISABLE_XM25QU128C
    {
        // XM25QU128C
        .mid = 0x184120,
        .capacity = 0x1000000,
        .mode = QUEC_NOR_FLASH_MODE_16BIT,
        .mount_start_addr = 0x0000000,
        .mount_size = 0x1000000,
    },
    #endif
    #ifndef DISABLE_W25Q64JWSSIM
    {
        // W25Q64JWSSIM
        .mid = 0x1780ef,
        .capacity = 0x800000,
        .mode = QUEC_NOR_FLASH_MODE_16BIT,
        .mount_start_addr = 0x0000000,
        .mount_size = 0x800000,
    },
    #endif
};

```

图 1：支持的 NOR Flash 型号相关参数信息

2.2. 相关参数含义

2.2.1. quec_spi_nor_flash_info_s

`quec_spi_nor_flash_props` (其类型 `quec_spi_nor_flash_info_s` 在 `quec_spi_nor_flash_prop.h` 中定义) 包含了 NOR flash 型号的相关参数，具体如下：

```
typedef struct
{
    unsigned int mid;
    int64 capacity;
    quec_spi_nor_flash_status_mode_e mode;
    unsigned int mount_start_addr;
    int32 mount_size;
}quec_spi_nor_flash_info_s
```

● 参数

类型	参数	描述
unsigned int	<i>mid</i>	通过 9Fh 命令读取的小端格式的 JEDEC ID，由厂商 ID（Manufacturer ID）和设备 ID（Device ID）构成。例如，JEDEC ID = 厂商 ID (M7–M0) + 设备 ID (ID15–ID0)，其小端格式为：(ID7–ID0) + (ID15–ID8) + (M7–M0)。
int64	<i>capacity</i>	容量大小。单位：字节。例如： 型号 XM25QU32C 大小为 32 Mbit，则该值为 0x400000； 型号 GD25LE64E 大小为 64 Mbit，则该值为 0x800000； 型号 XM25QU128C 大小为 128 Mbit，则该值为 0x1000000。
<i>quec_spi_nor_flash_status_mode_e</i>	<i>mode</i>	NOR flash 状态寄存器类型；详见第 2.2.1.1 章。
unsigned int	<i>mount_start_addr</i>	挂载文件系统时 NOR flash 的起始地址。需 4K 对齐。 如不挂载文件系统，则无需关注。
int32	<i>mount_size</i>	用于挂载文件系统的 NOR flash 分区的大小。需 4K 对齐。 如不挂载文件系统，则无需关注。

2.2.1.1. quec_spi_nor_flash_status_mode_e

NOR flash 状态寄存器类型枚举定义如下：

```
typedef enum
{
    QUEC_NOR_FLASH_MODE_8BIT = 0,
    QUEC_NOR_FLASH_MODE_16BIT,
```

```
}quec_spi_nor_flash_status_mode_e
```

- 成员

成员	描述
<i>QUEC_NOR_FLASH_MODE_8BIT</i>	8 位状态寄存器
<i>QUEC_NOR_FLASH_MODE_16BIT</i>	16 位或 24 位状态寄存器（24 位状态寄存器仅写入前 16 位）

2.2.1.2. 状态寄存器

状态寄存器的类型可通过 NOR flash 芯片数据手册获取。

示例一：如下图型号 XM25QU64B 数据手册中的状态寄存器定义所示，状态寄存器类型为 8 位。

	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
	SRWD	QE	BP3	BP2	BP1	BP0	WEL	WIP
Default	0	Note1	0	0	0	0	0	0

图 2：XM25QU64B 状态寄存器

示例二：如下图型号 GD25LQ64E 数据手册中的状态寄存器定义所示，2 个 8 位状态寄存器组成一个 16 位状态寄存器，状态寄存器类型为 16 位。

No.	Name	Description	Note
S7	SRP0	Status Register Protection Bit	Non-volatile writable
S6	BP4	Block Protect Bit	Non-volatile writable
S5	BP3	Block Protect Bit	Non-volatile writable
S4	BP2	Block Protect Bit	Non-volatile writable
S3	BP1	Block Protect Bit	Non-volatile writable
S2	BP0	Block Protect Bit	Non-volatile writable
S1	WEL	Write Enable Latch	Volatile, read only
S0	WIP	Erase/Write In Progress	Volatile, read only
No.	Name	Description	Note
S15	SUS1	Erase Suspend Bit	Volatile, read only
S14	CMP	Complement Protect Bit	Non-volatile writable
S13	LB3	Security Register Lock Bit	Non-volatile writable (OTP)
S12	LB2	Security Register Lock Bit	Non-volatile writable (OTP)
S11	LB1	Security Register Lock Bit	Non-volatile writable (OTP)
S10	SUS2	Program Suspend Bit	Volatile, read only
S9	QE	Quad Enable Bit	Non-volatile writable
S8	SRP1	Status Register Protection Bit	Non-volatile writable

图 3: GD25LQ64E 状态寄存器

示例三：如下图型号 XM25QU128C 数据手册中的命令表所示，3 个 8 位状态寄存器组成一个 24 位状态寄存器，状态寄存器类型为 24 位。

Data Input/Output	Byte 1	Byte 2	Byte 3	Byte 4	Byte 5	Byte 6	Byte 7
Clock Number	(0 – 7)	(8 – 15)	(16 – 23)	(24 – 31)	(32 – 39)	(40 – 47)	(48 – 55)
Write Enable	06h						
Volatile SR Write Enable	50h						
Write Disable	04h						
Read Status Register-1	05h	(S7-S0) ⁽²⁾					
Write Status Register-1 ⁽⁴⁾	01h	(S7-S0) ⁽⁴⁾					
Read Status Register-2	35h	(S15-S8) ⁽²⁾					
Write Status Register-2	31h	(S15-S8)					
Read Status Register-3	15h	(S23-S16) ⁽²⁾					
Write Status Register-3	11h	(S23-S16)					
Chip Erase	C7h/60h						

图 4: XM25QU128C 状态寄存器

3 SPI NOR Flash API

3.1. 4 线 SPI NOR Flash API

3.1.1. 头文件

4 线 SPI NOR flash API 头文件为 *ql_api_spi_nor_flash.h*, 位于 CSDK 包的 *components\ql-kernel\inc* 目录下。若无特别说明, 本文档所述头文件均在该目录下。

3.1.2. 函数概览

表 2: 函数概览

函数	说明
<i>ql_spi_nor_init()</i>	初始化配置 SPI (仅配置 SPI 总线和时钟频率)
<i>ql_spi_nor_init_ext()</i>	初始化配置 SPI
<i>ql_spi_nor_write_8bit_status()</i>	写入数据至 NOR flash 8 位状态寄存器
<i>ql_spi_nor_write_16bit_status()</i>	写入数据至 NOR flash 16 位或 24 位状态寄存器
<i>ql_spi_nor_read_status_low()</i>	读取 NOR flash 状态寄存器的低 8 位数据
<i>ql_spi_nor_read_status_high()</i>	读取 NOR flash 状态寄存器的高 8 位数据
<i>ql_spi_nor_read()</i>	读取 NOR flash 数据
<i>ql_spi_nor_write()</i>	写入数据至 NOR flash
<i>ql_spi_nor_erase_chip()</i>	对 NOR flash 进行整片擦除
<i>ql_spi_nor_erase_sector()</i>	擦除 NOR flash 指定地址的扇区 (4 KB 大小)
<i>ql_spi_nor_erase_64k_block()</i>	擦除 NOR flash 指定地址的块 (64 KB 大小)

3.1.3. 函数详解

3.1.3.1. ql_spi_nor_init

该函数用于初始化配置 SPI（仅配置 SPI 总线和时钟频率）。调用该函数前，需通过 `ql_pin_set_func()` 设置相关 GPIO 引脚为 SPI 功能，详情请参考文档 [2]。

调用该函数将默认配置 `ql_spi_input_sel_e` 为 `QL_SPI_DI_1`，`ql_spi_transfer_mode_e` 为 `QL_SPI_DIRECT_POLLING`，`ql_spi_cs_sel_e` 为 `QL_SPI_CS0`。相关参数描述详情请参考第 3.1.3.1.4 章、第 3.1.3.1.5 章和第 3.1.3.1.6 章。

- 函数原型

```
ql_errcode_spi_nor_e ql_spi_nor_init(ql_spi_port_e port, ql_spi_clk_e spiclk)
```

- 参数

port:

[In] SPI 总线；详见第 3.1.3.1.2 章。

spiclk:

[In] SPI 时钟频率；详见第 3.1.3.1.3 章。

- 返回值

详见第 3.1.3.1.1 章。

备注

若选择了错误的 SPI 总线进行初始化，将导致 NOR flash 读、写、擦除操作失败。因此，需对初始化返回值进行判定，若判定为初始化失败，应避免进行后续读、写、擦除操作。

3.1.3.1.1. ql_errcode_spi_nor_e

4 线 SPI NOR flash 结果码枚举定义如下：

```
typedef ql_errcode_spi_flash_e ql_errcode_spi_nor_e
```

```
typedef enum
```

```
{
```

```
    QL_SPI_FLASH_SUCCESS                = 0,
```

```
    QL_SPI_FLASH_ERROR                  = 1 |
```



```
(QL_COMPONENT_STORAGE_EXTFLASH << 16),
    QL_SPI_FLASH_PARAM_TYPE_ERROR,
    QL_SPI_FLASH_PARAM_DATA_ERROR,
    QL_SPI_FLASH_PARAM_ACQUIRE_ERROR,
    QL_SPI_FLASH_PARAM_NULL_ERROR,
    QL_SPI_FLASH_DEV_NOT_ACQUIRE_ERROR,
    QL_SPI_FLASH_PARAM_LENGTH_ERROR,
    QL_SPI_FLASH_MALLOC_MEM_ERROR,
    QL_SPI_FLASH_NOT_FLASH_CONN_ERROR,
    QL_SPI_FLASH_NOT_SUPPORT_ERROR,
    QL_SPI_FLASH_OP_NOT_SUPPORT_ERROR,
    QL_SPI_FLASH_ECC_ERROR,
    QL_SPI_FLASH_PROGRAM_ERROR,
    QL_SPI_FLASH_ERASE_ERROR,
    QL_SPI_FLASH_MUTEX_CREATE_ERROR,
    QL_SPI_FLASH_MUTEX_LOCK_ERROR,
    QL_SPI_FLASH_SET_GPIO_ERROR,
}ql_errcode_spi_flash_e
```

● 成员

成员	描述
<code>QL_SPI_FLASH_SUCCESS</code>	函数执行成功
<code>QL_SPI_FLASH_ERROR</code>	SPI 总线其他错误
<code>QL_SPI_FLASH_PARAM_TYPE_ERROR</code>	参数类型错误
<code>QL_SPI_FLASH_PARAM_DATA_ERROR</code>	参数数据错误
<code>QL_SPI_FLASH_PARAM_ACQUIRE_ERROR</code>	参数无法获取
<code>QL_SPI_FLASH_PARAM_NULL_ERROR</code>	参数 NULL 错误
<code>QL_SPI_FLASH_DEV_NOT_ACQUIRE_ERROR</code>	无法获取 SPI 总线
<code>QL_SPI_FLASH_PARAM_LENGTH_ERROR</code>	参数长度错误
<code>QL_SPI_FLASH_MALLOC_MEM_ERROR</code>	申请内存错误
<code>QL_SPI_FLASH_NOT_FLASH_CONN_ERROR</code>	未接入 flash 芯片
<code>QL_SPI_FLASH_NOT_SUPPORT_ERROR</code>	flash 型号不支持
<code>QL_SPI_FLASH_OP_NOT_SUPPORT_ERROR</code>	flash 型号不支持此操作
<code>QL_SPI_FLASH_ECC_ERROR</code>	NAND flash ECC 校验错误（仅适用外接 NAND flash）

QL_SPI_FLASH_PROGRAM_ERROR	flash 程序错误
QL_SPI_FLASH_ERASE_ERROR	flash 擦除错误
QL_SPI_FLASH_MUTEX_CREATE_ERROR	flash 互斥锁创建失败错误
QL_SPI_FLASH_MUTEX_LOCK_ERROR	flash 互斥锁上锁超时错误
QL_SPI_FLASH_SET_GPIO_ERROR	flash 设置引脚错误

3.1.3.1.2. ql_spi_port_e

SPI 总线枚举定义如下：

```
typedef enum
{
    QL_SPI_PORT1,
    QL_SPI_PORT2,
}ql_spi_port_e
```

● 成员

成员	描述
QL_SPI_PORT1	SPI1 总线
QL_SPI_PORT2	SPI2 总线

3.1.3.1.3. ql_spi_clk_e

ECx00U、EGx00U 和 EG91xU 系列模块的 SPI 时钟频率枚举定义如下：

```
typedef enum
{
    QL_SPI_CLK_INVALID=-1,
    QL_SPI_CLK_781_25KHZ = 781250,
    QL_SPI_CLK_1_5625MHZ = 1562500,
    QL_SPI_CLK_3_125MHZ = 3125000,
    QL_SPI_CLK_5MHZ = 5000000,
    QL_SPI_CLK_6_25MHZ = 6250000,
    QL_SPI_CLK_10MHZ = 10000000,
    QL_SPI_CLK_12_5MHZ = 12500000,
    QL_SPI_CLK_20MHZ = 20000000,
    QL_SPI_CLK_25MHZ = 25000000,
    QL_SPI_CLK_33_33MHZ = 33000000,
```

```
QL_SPI_CLK_50MHZ_MAX = 50000000,
}ql_spi_clk_e
```

● 成员

成员	描述
QL_SPI_CLK_INVALID	无效参数
QL_SPI_CLK_781_25KHZ	时钟频率为 781.25 kHz
QL_SPI_CLK_1_5625MHZ	时钟频率为 1.5625 MHz
QL_SPI_CLK_3_125MHZ	时钟频率为 3.125 MHz
QL_SPI_CLK_5MHZ	时钟频率为 5 MHz
QL_SPI_CLK_6_25MHZ	时钟频率为 6.25 MHz
QL_SPI_CLK_10MHZ	时钟频率为 10 MHz
QL_SPI_CLK_12_5MHZ	时钟频率为 12.5 MHz
QL_SPI_CLK_20MHZ	时钟频率为 20 MHz
QL_SPI_CLK_25MHZ	时钟频率为 25 MHz
QL_SPI_CLK_33_33MHZ	时钟频率为 33.33 MHz
QL_SPI_CLK_50MHZ_MAX	时钟频率为 50 MHz（最大时钟频率）

ECx00G 和 EGx00G 系列模块的 SPI 时钟频率枚举定义如下：

```
typedef enum
{
    QL_SPI_CLK_INVALID=-1,
    QL_SPI_CLK_97_656KHZ_MIN = 97656,
    QL_SPI_CLK_100KHZ = 100000,
    QL_SPI_CLK_812_5KHZ = 812500,
    QL_SPI_CLK_1_3MHZ = 1300000,
    QL_SPI_CLK_1_625MHZ = 1625000,
    QL_SPI_CLK_1_5625MHZ = QL_SPI_CLK_1_625MHZ,
    QL_SPI_CLK_2MHZ = 2000000,
    QL_SPI_CLK_3_25MHZ = 3250000,
    QL_SPI_CLK_4_333MHZ = 4333000,
    QL_SPI_CLK_6_6MHZ = 6600000,
    QL_SPI_CLK_11_93MHZ = 11930000,
    QL_SPI_CLK_13MHZ = 13000000,
```

```

QL_SPI_CLK_13_92MHZ = 13920000,
QL_SPI_CLK_16_7MHZ = 16700000,
QL_SPI_CLK_20_875MHZ = 20875000,
QL_SPI_CLK_27_83MHZ = 27830000,
QL_SPI_CLK_25MHZ = QL_SPI_CLK_27_83MHZ,
QL_SPI_CLK_41_75MHZ = 41750000,
QL_SPI_CLK_50MHZ_MAX = 50000000,
}ql_spi_clk_e

```

● 成员

成员	描述
<code>QL_SPI_CLK_INVALID</code>	无效参数
<code>QL_SPI_CLK_97_656KHZ_MIN</code>	时钟频率为 97.656 kHz
<code>QL_SPI_CLK_100KHZ</code>	时钟频率为 100 kHz
<code>QL_SPI_CLK_812_5KHZ</code>	时钟频率为 812.5 kHz
<code>QL_SPI_CLK_1_3MHZ</code>	时钟频率为 1.3 MHz
<code>QL_SPI_CLK_1_625MHZ</code>	时钟频率为 1.625 MHz
<code>QL_SPI_CLK_1_5625MHZ</code>	时钟频率为 1.625 MHz
<code>QL_SPI_CLK_2MHZ</code>	时钟频率为 2 MHz
<code>QL_SPI_CLK_3_25MHZ</code>	时钟频率为 3.25 MHz
<code>QL_SPI_CLK_4_333MHZ</code>	时钟频率为 4.333 MHz
<code>QL_SPI_CLK_6_6MHZ</code>	时钟频率为 6.6 MHz
<code>QL_SPI_CLK_11_93MHZ</code>	时钟频率为 11.93 MHz
<code>QL_SPI_CLK_13MHZ</code>	时钟频率为 13 MHz
<code>QL_SPI_CLK_13_92MHZ</code>	时钟频率为 13.92 MHz
<code>QL_SPI_CLK_16_7MHZ</code>	时钟频率为 16.7 MHz
<code>QL_SPI_CLK_20_875MHZ</code>	时钟频率为 20.875 MHz
<code>QL_SPI_CLK_27_83MHZ</code>	时钟频率为 27.83 MHz
<code>QL_SPI_CLK_25MHZ</code>	时钟频率为 27.83 MHz
<code>QL_SPI_CLK_41_75MHZ</code>	时钟频率为 41.75 MHz
<code>QL_SPI_CLK_50MHZ_MAX</code>	时钟频率为 50 MHz（最大时钟频率）

3.1.3.1.4. ql_spi_input_sel_e

数据输入引脚枚举定义如下：

```
typedef enum
{
    QL_SPI_DI_0 = 0,
    QL_SPI_DI_1,
    QL_SPI_DI_2,
}ql_spi_input_sel_e
```

● 成员

成员	描述
QL_SPI_DI_0	DI0 为数据输入引脚（暂不支持）
QL_SPI_DI_1	DI1 为数据输入引脚
QL_SPI_DI_2	DI2 为数据输入引脚（暂不支持）

3.1.3.1.5. ql_spi_transfer_mode_e

SPI 传输模式枚举定义如下：

```
typedef enum
{
    QL_SPI_DIRECT_POLLING = 0,
    QL_SPI_DIRECT_IRQ,
    QL_SPI_DMA_POLLING,
    QL_SPI_DMA_IRQ,
}ql_spi_transfer_mode_e
```

● 成员

成员	描述
QL_SPI_DIRECT_POLLING	FIFO 读写，轮询等待。
QL_SPI_DIRECT_IRQ	FIFO 读写，中断通知（暂不支持）。
QL_SPI_DMA_POLLING	DMA 读写，轮询等待。
QL_SPI_DMA_IRQ	DMA 读写，中断通知（暂不支持）。

备注

ECx00U、EGx00U 和 EG91xU 系列模块使用 DMA 轮询模式时，SPI 与 SD 卡抢占 DMA 通道；若 SD 卡已使用 DMA 通道，SPI 初始化则会失败。ECx00G 和 EGx00G 系列模块无此影响。

3.1.3.1.6. ql_spi_cs_sel_e

CS 引脚枚举定义如下：

```
typedef enum
{
    QL_SPI_CS0 = 0,
    QL_SPI_CS1,
    QL_SPI_CS2,
    QL_SPI_CS3,
}ql_spi_cs_sel_e
```

● 成员

成员	描述
<i>QL_SPI_CS0</i>	CS0 为 SPI 的 CS 引脚
<i>QL_SPI_CS1</i>	CS1 为 SPI 的 CS 引脚
<i>QL_SPI_CS2</i>	CS2 为 SPI 的 CS 引脚（暂不支持）
<i>QL_SPI_CS3</i>	CS3 为 SPI 的 CS 引脚（暂不支持）

3.1.3.2. ql_spi_nor_init_ext

该函数用于初始化配置 SPI，包括 SPI 总线、时钟频率、传输模式、数据输入引脚和 CS 引脚。调用该函数前，需调用 *ql_pin_set_func()* 设置相关 GPIO 引脚为 SPI 功能，详情请参考 [文档 \[2\]](#)。

● 函数原型

```
ql_errcode_spi_nor_e ql_spi_nor_init_ext(ql_spi_nor_config_s nor_config)
```

● 参数

nor_config:

[In] SPI 配置参数；详见 [第 3.1.3.2.1 章](#)。

- 返回值

详见第3.1.3.1.1章。

备注

若选择了错误的 SPI 总线进行初始化，将导致 NOR flash 读、写、擦除操作失败。因此，需对初始化返回值进行判定，若判定为初始化失败，应避免进行后续读、写、擦除操作。

3.1.3.2.1. ql_spi_nor_config_s

SPI 配置参数结构体定义如下：

```
typedef ql_spi_flash_config_s ql_spi_nor_config_s
```

```
typedef struct
{
    ql_spi_port_e port;
    ql_spi_clk_e spiclk;
    ql_spi_input_sel_e input_sel;
    ql_spi_transfer_mode_e transmode;
    ql_spi_cs_sel_e cs;
} ql_spi_flash_config_s
```

- 参数

类型	参数	描述
<i>ql_spi_port_e</i>	<i>port</i>	SPI 总线；详见第3.1.3.1.2章。
<i>ql_spi_clk_e</i>	<i>spiclk</i>	SPI 时钟频率；详见第3.1.3.1.3章。
<i>ql_spi_input_sel_e</i>	<i>input_sel</i>	数据输入引脚；详见第3.1.3.1.4章。
<i>ql_spi_transfer_mode_e</i>	<i>transmode</i>	SPI 传输模式；详见第3.1.3.1.5章。
<i>ql_spi_cs_sel_e</i>	<i>cs</i>	CS 引脚；详见第3.1.3.1.6章。

3.1.3.3. ql_spi_nor_write_8bit_status

该函数用于写入数据至 NOR flash 8 位状态寄存器。

- 函数原型

```
ql_errcode_spi_nor_e ql_spi_nor_write_8bit_status(ql_spi_port_e port, unsigned char status)
```

- 参数

port:

[In] SPI 总线；详见第3.1.3.1.2 章。

status:

[In] 需写入 NOR flash 状态寄存器的数据。

- 返回值

详见第3.1.3.1.1 章。

3.1.3.4. ql_spi_nor_write_16bit_status

该函数用于写入数据至 NOR flash 16 位或 24 位状态寄存器。24 位状态寄存器仅写入前 16 位。

- 函数原型

```
ql_errcode_spi_nor_e ql_spi_nor_write_16bit_status(ql_spi_port_e port, unsigned short status)
```

- 参数

port:

[In] SPI 总线；详见第3.1.3.1.2 章。

status:

[In] 需写入 NOR flash 状态寄存器中的数据。

- 返回值

详见第3.1.3.1.1 章。

3.1.3.5. ql_spi_nor_read_status_low

该函数用于读取 NOR flash 状态寄存器的低 8 位数据。

- 函数原型

```
ql_errcode_spi_nor_e ql_spi_nor_read_status_low(ql_spi_port_e port, unsigned char *status)
```

- 参数

port:

[In] SPI 总线；详见第 3.1.3.1.2 章。

status:

[Out] 读取的状态寄存器中的低 8 位数据。

- 返回值

详见第 3.1.3.1.1 章。

3.1.3.6. ql_spi_nor_read_status_high

该函数用于读取 NOR flash 状态寄存器的高 8 位数据。

- 函数原型

```
ql_errcode_spi_nor_e ql_spi_nor_read_status_high(ql_spi_port_e port, unsigned char *status)
```

- 参数

port:

[In] SPI 总线；详见第 3.1.3.1.2 章。

status:

[Out] 读取的状态寄存器中的高 8 位数据。

- 返回值

详见第 3.1.3.1.1 章。

3.1.3.7. ql_spi_nor_read

该函数用于读取 NOR flash 数据。

- 函数原型

```
ql_errcode_spi_nor_e ql_spi_nor_read(ql_spi_port_e port, unsigned char* data, unsigned int addr, unsigned short len)
```

- 参数

port:

[In] SPI 总线；详见第3.1.3.1.2 章。

data:

[Out] 读取数据。

addr:

[In] 读取数据的地址。

len:

[In] 读取数据长度。单位：字节。

- 返回值

详见第3.1.3.1.1 章。

3.1.3.8. ql_spi_nor_write

该函数用写入数据至 NOR flash。

- 函数原型

```
ql_errcode_spi_nor_e ql_spi_nor_write(ql_spi_port_e port, unsigned char *data, unsigned int addr, unsigned short len)
```

- 参数

port:

[In] SPI 总线；详见第3.1.3.1.2 章。

data:

[In] 写入数据。

addr:

[In] 写入数据的地址。

len:

[In] 写入数据长度。单位：字节。

- 返回值

详见第3.1.3.1.1章。

3.1.3.9. ql_spi_nor_erase_chip

该函数用于对 NOR flash 进行整片擦除。

- 函数原型

```
ql_errcode_spi_nor_e ql_spi_nor_erase_chip(ql_spi_port_e port)
```

- 参数

port:

[In] SPI 总线；详见第3.1.3.1.2章。

- 返回值

详见第3.1.3.1.1章。

3.1.3.10. ql_spi_nor_erase_sector

该函数用于擦除 NOR flash 指定地址的扇区（4 KB 大小）。

- 函数原型

```
ql_errcode_spi_nor_e ql_spi_nor_erase_sector(ql_spi_port_e port, unsigned int addr)
```

- 参数

port:

[In] SPI 总线；详见第3.1.3.1.2章。

addr:

[In] 擦除扇区的地址。

- 返回值

详见第3.1.3.1.1章。

3.1.3.11. ql_spi_nor_erase_64k_block

该函数用于擦除 NOR flash 指定地址的块（64 KB 大小）。

- 函数原型

```
ql_errcode_spi_nor_e ql_spi_nor_erase_64k_block(ql_spi_port_e port, unsigned int addr)
```

- 参数

port:

[In] SPI 总线；详见第3.1.3.1.2章。

addr:

[In] 擦除块的地址。

- 返回值

详见第3.1.3.1.1章。

3.2. 4 线 SPI NOR Flash 文件系统挂载 API

3.2.1. 头文件

4 线 SPI NOR flash 文件系统挂载 API 头文件为 *ql_api_spi4_ext_nor_sffs.h*，位于 CSDK 包的 *components\ql-kernel\inc* 目录下。若无特别说明，本文档所述头文件均在该目录下。

3.2.2. 函数概览

表 3：函数概览

函数	说明
<i>ql_spi4_ext_nor_sffs_set_port()</i>	设置 SFFS 文件系统中使用的 SPI 总线
<i>ql_spi4_ext_nor_sffs_get_port()</i>	获取 SFFS 文件系统中使用的 SPI 总线
<i>ql_spi4_ext_nor_sffs_mount()</i>	挂载 4 线 SPI NOR flash 到 SFFS 文件系统

<code>ql_spi4_ext_nor_sffs_unmount()</code>	从 SFFS 文件系统中卸载 4 线 SPI NOR flash
<code>ql_spi4_ext_nor_sffs_mkfs()</code>	将 4 线 SPI NOR flash 格式化为 SFFS 文件系统格式
<code>ql_spi4_ext_nor_sffs_is_mount()</code>	查询 4 线 SPI NOR flash 是否已挂载到 SFFS 文件系统

3.2.3. 函数详解

3.2.3.1. ql_spi4_ext_nor_sffs_set_port

该函数用于设置 SFFS 文件系统中使用的 SPI 总线。初始化 4 线 SPI NOR Flash 后，必须调用此接口设置当前使用的 SPI 端口，否则使用默认的 `QL_SPI_PORT1`。

- 函数原型

```
void ql_spi4_ext_nor_sffs_set_port(ql_spi_port_e spi_port)
```

- 参数

port:

[In] SPI 总线；详情请参考文档 [3]。

- 返回值

无

3.2.3.2. ql_spi4_ext_nor_sffs_get_port

该函数用于获取 SFFS 文件系统中使用的 SPI 总线。

- 函数原型

```
ql_spi_port_e ql_spi4_ext_nor_sffs_get_port(void)
```

- 参数

无

- 返回值

SPI 总线；详情请参考文档 [3]。

3.2.3.3. ql_spi4_ext_nor_sffs_mount

该函数用于挂载 4 线 SPI NOR flash 到 SFFS 文件系统。

● 函数原型

```
ql_errcode_spi4_extnsffs_e ql_spi4_ext_nor_sffs_mount(void)
```

● 参数

无

● 返回值

详见第 3.2.3.3.1 章。

3.2.3.3.1. ql_errcode_spi4_extnsffs_e

4 线 SPI NOR flash 文件系统挂载结果码枚举定义如下：

```
typedef enum
{
    QL_SPI4_EXT_NOR_SFFS_SUCCESS = 0,
    QL_SPI4_EXT_NOR_SFFS_NOT_INIT_ERROR = 100 |
(QL_COMPONENT_STORAGE_EXTFLASH << 16),
    QL_SPI4_EXT_NOR_SFFS_CREATE_FBDEV2SPI_ERROR,
    QL_SPI4_EXT_NOR_SFFS_SFFS_MOUNT_ERROR,
    QL_SPI4_EXT_NOR_SFFS_SFFS_UNMOUNT_ERROR,
    QL_SPI4_EXT_NOR_SFFS_SFFS_MKFS_ERROR,
    QL_SPI4_EXT_NOR_SFFS_LOCK_ERROR,
}ql_errcode_spi4_extnsffs_e
```

● 成员

成员	描述
QL_SPI4_EXT_NOR_SFFS_SUCCESS	函数执行成功
QL_SPI4_EXT_NOR_SFFS_NOT_INIT_ERROR	SPI NOR flash 初始化错误
QL_SPI4_EXT_NOR_SFFS_CREATE_FBDEV2SPI_ERROR	创建块设备分区错误
QL_SPI4_EXT_NOR_SFFS_SFFS_MOUNT_ERROR	挂载错误
QL_SPI4_EXT_NOR_SFFS_SFFS_UNMOUNT_ERROR	卸载错误

QL_SPI4_EXT_NOR_SFFS_SFFS_MKFS_ERROR	格式化错误
QL_SPI4_EXT_NOR_SFFS_LOCK_ERROR	互斥锁上锁错误

3.2.3.4. ql_spi4_ext_nor_sffs_unmount

该函数用于从 SFFS 文件系统中卸载 4 线 SPI NOR flash。

- 函数原型

```
ql_errcode_spi4_extnsffs_e ql_spi4_ext_nor_sffs_unmount(void)
```

- 参数

无

- 返回值

详见第 3.2.3.3.1 章。

3.2.3.5. ql_spi4_ext_nor_sffs_mkfs

该函数用于将 4 线 SPI NOR flash 格式化为 SFFS 文件系统格式。

- 函数原型

```
ql_errcode_spi4_extnsffs_e ql_spi4_ext_nor_sffs_mkfs(void)
```

- 参数

无

- 返回值

详见第 3.2.3.3.1 章。

3.2.3.6. ql_spi4_ext_nor_sffs_is_mount

该函数用于查询 4 线 SPI NOR flash 是否已挂载到 SFFS 文件系统。

- 函数原型

```
ql_spi4_extnsffs_status_e ql_spi4_ext_nor_sffs_is_mount(void)
```

- 参数

无

- 返回值

详见第 3.2.3.6.1 章。

3.2.3.6.1. ql_spi4_extnsffs_status_e

4 线 SPI NOR flash 状态枚举定义如下：

```
typedef enum
{
    QL_SPI4_EXT_NOR_SFFS_IS_UNMOUNTED = 0,
    QL_SPI4_EXT_NOR_SFFS_IS_MOUNTED
}ql_spi4_extnsffs_status_e
```

- 成员

成员	描述
<i>QL_SPI4_EXT_NOR_SFFS_IS_UNMOUNTED</i>	未挂载文件系统
<i>QL_SPI4_EXT_NOR_SFFS_IS_MOUNTED</i>	已挂载文件系统

4 示例

本章节主要介绍在应用层如何使用上述 API 进行外接 4 线 SPI NOR flash 和 SPI NOR flash 文件系统挂载开发以及简单调试。

4.1. 开发示例

4.1.1. 4 线 SPI NOR Flash API 开发示例

模块的 QuecOpen CSDK 中提供了 4 线 SPI NOR flash API 示例文件 `components\ql-application\spi_nor_flash\spi_nor_flash_demo.c`。该示例中主要包含 SPI NOR flash 初始化、读取和写入数据、擦除扇区、擦除块及整片擦除等操作。示例入口函数为 `ql_spi_nor_flash_demo_init()`，如下图所示。

```
QLOSStatus ql_spi_nor_flash_demo_init(void)
{
    QLOSStatus err = QL_OSI_SUCCESS;
    err = ql_rtos_task_create(&spi_nor_flash_demo_task, SPI_NOR_FLASH_DEMO_TASK_STACK_SIZE, SPI_NOR_FLASH_DEMO_TASK_PRIO, "ql_spi_nor", ql_spi_nor_flash_demo_task_pthread, NULL, SPI_NOR_FLASH_DEMO_TASK_EVENT_CNT);
    if(err != QL_OSI_SUCCESS)
    {
        QL_SPI_NOR_LOG("demo_task created failed");
        return err;
    }
    return err;
}
```

图 5：入口函数 `ql_spi_nor_flash_demo_init()`

如下图所示，该示例已在 `ql_init_demo_thread` 线程中默认不启动。如有需要，取消 `ql_spi_nor_flash_demo_init()` 的注释即可运行。

```
#ifndef QL_APP_FEATURE_SPI_NOR_FLASH
    //ql_spi_nor_flash_demo_init();
#endif
```

图 6：4 线 SPI NOR Flash API 示例默认不启动

4.1.2. 4 线 SPI NOR Flash 文件系统挂载 API 开发示例

模块的 QuecOpen CSDK 中提供了 4 线 SPI NOR flash 文件系统挂载 API 示例文件 `components\ql-application\spi4_ext_nor_sffs\spi4_ext_nor_sffs_demo.c`。该示例中主要包含 SPI NOR flash 初始化、挂载 SFFS 文件系统、格式化 SPI NOR flash、通过文件系统 API 进行读写及卸载 SFFS 文件系统等操作。示例入口函数为 `ql_spi4_ext_nor_sffs_demo_init()`，如下图所示。

```

Q1OSStatus ql_spi4_ext_nor_sffs_demo_init(void)
{
    Q1OSStatus err = QL_OSI_SUCCESS;

    ...//QL_SPI4_EXTNSFFS_LOG("enter ql_spi4_ext_nor_sffs_demo_init");
    err = ql_rtos_task_create(&spi4_ext_nor_sffs_demo_task,
        SPI4_EXT_NOR_SFFS_DEMO_TASK_STACK_SIZE,
        SPI4_EXT_NOR_SFFS_DEMO_TASK_PRIO,
        "ql_spi4_sffs",
        ql_spi4_ext_nor_sffs_demo_task_pthread,
        NULL,
        SPI4_EXT_NOR_SFFS_DEMO_TASK_EVENT_CNT);
    if(err != QL_OSI_SUCCESS)
    {
        QL_SPI4_EXTNSFFS_LOG("demo_task created failed");
        ...return err;
    }

    ...return err;
} //end ql_spi4_ext_nor_sffs_demo_init

```

图 7: 入口函数 `ql_spi4_ext_nor_sffs_demo_init()`

如下图所示，该示例已在 `ql_init_demo_thread` 线程中默认不启动。如有需要，取消 `ql_spi4_ext_nor_sffs_demo_init()` 的注释即可运行。

```

#ifdef QL_APP_FEATURE_SPI4_EXT_NOR_SFFS
    ...//ql_spi4_ext_nor_sffs_demo_init();
#endif

```

图 8: 4 线 SPI NOR Flash 文件系统挂载 API 示例默认不启动

4.2. 功能调试

4.2.1. 4 线 SPI NOR Flash API 功能调试

调试 SPI NOR flash 功能前，需在移远通信的 LTE OPEN EVB 上外接 NOR flash 芯片。

编译固件版本烧录到模块中，使用 USB 线连接 LTE OPEN EVB 板的 USB 端口和 PC，USB AP Log Port 和 USB DIAG Port 主要用于显示系统调试信息。通过 cooltools 或 ArmLogel 工具抓取 log 后，可以查看上述应用示例的相关信息，log 的抓取方法请参考文档 [4] 和 [5]。



图 9: COM 设备图 1 (ECx00U 系列/EGx00U 系列/EG91xU 系列模块)



4.2.2. 4 线 SPI NOR Flash 文件系统挂载 API 功能调试

调试 4 线 SPI NOR flash 功能前，需在移远通信的 LTE OPEN EVB 上外接 NOR flash 芯片。

编译固件版本烧录到模块中，使用 USB 线连接 LTE OPEN EVB 板的 USB 端口和 PC，USB AP Log Port 和 USB DIAG Port 主要用于显示系统调试信息。通过 cooltools 或 ArmLogel 工具抓取 log 后，可以查看上述应用示例的相关信息，log 的抓取方法请参考文档 [4]和[5]。不同模块的 COM 设备图，见图9 和图10。

模块开机后自动启动 `ql_spi4_ext_nor_sffs_demo_init()`。通过 log 信息可以看到 4 线 SPI NOR flash 文件系统挂载的相关信息。USB AP Log Port 调试信息如下图所示。

```
[16:47:51.142] [19877] [ 1169] [ ] [ ] [D] FSYS/I : FBDEV: scan EB quickinit/0 invalid/1 samelb/0 erase count/3/3 next use/20
[16:47:51.142] [19877] [ 1170] [ ] [ ] [D] FSYS/I : SFFS mount, device block size/500 block count/16255 cache count 4
[16:47:51.168] [20038] [ 1171] [ ] [ ] [D] FSYS/I : SFFS mount move count/0 block free/8067
[16:47:51.168] [20337] [ 1172] [ ] [ ] [ ] QOPN/I : [ql_SPI4_SFFS_DEMO][ql_spi4_ext_nor_sffs_demo_task_pthread, 160] sffs mount succeed
[16:47:51.168] [20343] [ 1173] [ ] [ ] [ ] QOPN/I : [ql_SPI4_SFFS_DEMO][ql_spi4_ext_nor_sffs_demo_task_pthread, 170] fs free_size :3964892
[16:47:51.168] [20345] [ 1174] [ ] [ ] [ ] QOPN/I : [ql_fs_hd][ql_get_file_info, 687] get node count failed, or no file in list
[16:47:51.233] [20372] [ 1175] [ ] [ ] [ ] FSYS/I : prvFlashErase,offset=0xa0000,size=0x8000,phy_size=0x7f8000
[16:47:51.510] [23439] [ 1494] [ ] [ ] [ ] QOPN/I : [ql_fs][ql_fopen, 169] open file EXNSFFS:test.txt, fd 5123
[16:47:51.510] [23445] [ 1495] [ ] [ ] [ ] QOPN/I : [ql_fs][ql_fseek, 438] ret:0
[16:47:51.510] [23448] [ 1496] [ ] [ ] [ ] QOPN/I : [ql_SPI4_SFFS_DEMO][ql_spi4_ext_nor_sffs_demo_task_pthread, 204] file read result is 1234567890abodefg
[16:47:51.516] [25022] [ 1503] [ ] [ ] [ ] QOPN/I : [ql_fs][ql_fclose, 264] close file fd 5123
[16:47:51.516] [25023] [ 1504] [ ] [ ] [ ] QOPN/I : [ql_SPI4_SFFS_DEMO][ql_spi4_ext_nor_sffs_demo_task_pthread, 212] sffs unmount succeed
[16:47:51.516] [25026] [ 1505] [ ] [ ] [ ] QOPN/I : [ql_fs_hd][ql_get_file_info, 687] get node count failed, or no file in list
[16:47:51.516] [25028] [ 1506] [ ] [ ] [ ] QOPN/I : [ql_fs][ql_fopen, 119] get free size failed
[16:47:51.516] [25028] [ 1507] [ ] [ ] [ ] QOPN/I : [ql_SPI4_SFFS_DEMO][ql_spi4_ext_nor_sffs_demo_task_pthread, 216] open file failed,errcode is 830301a3
[16:47:51.589] [25029] [ 1508] [ ] [ ] [D] FSYS/I : FBDEV: create phys_start/0x0 phys_size/0x7f8000 eb_size/0x8000 pb_size/0x200 read_only/0
[16:47:56.085] [35466] [ 1910] [ ] [ ] [D] FSYS/I : FBDEV: scan EB quickinit/0 invalid/1 samelb/0 erase count/3/3 next use/148
[16:47:56.108] [35466] [ 1911] [ ] [ ] [D] FSYS/I : SFFS mount, device block size/500 block count/16255 cache count 4
[16:47:56.108] [35631] [ 1912] [ ] [ ] [D] FSYS/I : SFFS mount move count/0 block free/8066
[16:47:56.108] [35753] [ 1913] [ ] [ ] [ ] QOPN/I : [ql_SPI4_SFFS_DEMO][ql_spi4_ext_nor_sffs_demo_task_pthread, 225] sffs remount succeed
[16:47:56.108] [35758] [ 1914] [ ] [ ] [ ] QOPN/I : [ql_fs_hd][ql_get_file_info, 687] get node count failed, or no file in list
[16:47:56.217] [37317] [ 1921] [ ] [ ] [ ] QOPN/I : [ql_fs][ql_fopen, 169] open file EXNSFFS:test.txt, fd 5123
[16:47:56.217] [37319] [ 1922] [ ] [ ] [ ] QOPN/I : [ql_fs][ql_fclose, 264] close file fd 5123
[16:47:56.217] [37319] [ 1923] [ ] [ ] [ ] QOPN/I : [ql_SPI4_SFFS_DEMO][ql_spi4_ext_nor_sffs_demo_task_pthread, 236] close file
[16:47:56.217] [37320] [ 1924] [ ] [ ] [ ] QOPN/I : [ql_SPI4_SFFS_DEMO][ql_spi4_ext_nor_sffs_demo_task_pthread, 249] ql_rtos_task_delete
[16:47:56.267] [37321] [ 1925] [ ] [ ] [ ] QOPN/I : [ql_OS][ql_rtos_task_delete, 183] remove task 80C0B3A0 ok
```

图 13: 4 线 SPI NOR Flash 文件系统挂载 Log 信息

格式化时，若出现如下图所示的 dump log，可能是由于 NOR flash 被写保护，导致数据不能正常写入 NOR flash。

11:48:21.128	7387	QOPN/I	[ql_SPI_NOR][ql_spi_nor_read_id, 622] device id=17
11:48:21.128	7388	QOPN/I	[ql_SPI_NOR][ql_spi_nor_read_devid, 651] mid =184120
11:48:21.128	7389	QUEV/I	[ql_SPI4SFFS][drvGeneralSpiFlashOpen, 144] Flash 0x0,0x400000,0x1000000
11:48:21.128	7389	FSYS/I	FBDEV: create phys_start/0x0 phys_size/0x3ff000 eb_size/0x1000 pb_size/0x200 read_only/0
11:48:21.128	7604	QOPN/I	[ql_LEDCFGDEMO][ql_ledcfg_demo_thread, 165] led config slow twinkle [0]
11:48:21.128	7604	QOPN/I	[ql_PWM][ql_pwm_lpg_enable, 147] LPG enable OK!
11:48:21.209	7604	QOPN/I	[ql_LEDCFGDEMO][ql_ledcfg_demo_thread, 200] led config network is not 4G
11:48:21.338	11908	FSYS/I	FBDEV: scan EB quickinit/0 invalid/1023 samelb/0 erase count/15728640/0 next use/0
11:48:21.338	11909	FSYS/I	SFFS mount, device block size/500 block count/8175 cache count 4
11:48:21.338	11910	FSYS/E	SFFS mount wrong root tag
11:48:21.338	11910	FSYS/I	prvMountSffs mount failed!
11:48:21.393	11911	FSYS/I	FBDEV: create phys_start/0x0 phys_size/0x3ff000 eb_size/0x1000 pb_size/0x200 read_only/0
11:48:21.604			Detected event: 0x00a55e47
11:48:21.604			Detected event: 0x9db10000
11:48:21.712	16417	FSYS/I	FBDEV: scan EB quickinit/0 invalid/1023 samelb/0 erase count/15728640/0 next use/0
11:48:21.822			Detected event: 0x9db00000
11:48:22.052	16536	FSYS/I	prvFlashErase,offset=0x0,size=0x1000, phy_size=0x3ff000
11:48:22.052	16537	QUEV/I	[ql_SPI4SFFS][drvGeneralSpiFlashFinish, 162] FlashFinish read 0x5
11:48:22.052	16539	QUEV/I	[ql_SPI4SFFS][drvGeneralSpiFlashFinish, 162] FlashFinish read 0x5
11:48:22.052	16539	QUEC/I	drvGeneralSpiFlashErase Success
11:48:22.052	16539	QUEC/I	drvGeneralSpiFlashErase 0x1000,0x0
11:48:22.052	16547	FSYS/I	FBDEV: prvFlash Write Enter, offset=0x0, data=0x80bfd6b8, size=0x200, phy_size=0x3ff000
11:48:22.052	16550	QUEV/I	[ql_SPI4SFFS][drvGeneralSpiFlashFinish, 162] FlashFinish read 0x5
11:48:22.052	16552	QUEV/I	[ql_SPI4SFFS][drvGeneralSpiFlashFinish, 162] FlashFinish read 0x5
11:48:22.052	16552	QUEC/I	drvGeneralSpiFlashWrite 0x200,0x0
11:48:22.052	16553	FSYS/I	FBDEV: prvFlash Write Exit
11:48:22.052	16553	FSYS/I	prvFlashWriteCheck, offset=0x0,data=0x80bfd6b8,size=0x200,
11:48:22.052	16557	KERN/E	!!! PANIC AT 6009AAAB
11:48:22.052	16558	IPCD/E	!!! CP PANIC:
11:48:22.052	16561	/E	!!! BLUE SCREEN pc/0x800b6a lr/0x800b6b sp/0x80bfcf30 cpsr/0x200701db spsr/0x2007003f
11:48:22.052	23194	KERN/I	TASK count 58
11:48:22.052	23194	KERN/I	TASK 58 (ql_spi4_sffs) state/0 prio/12/12

图 14: Dump Log 信息

此时，需要根据 `quec_spi_nor_flash_info_s` 中 `mode` 参数取值，选择使用对应的函数来清除状态寄存器写保护位。

5 附录 参考文档及术语缩写

表 4: 参考文档

文档名称
[1] Quectel_LTE_Standard(U)系列_QuecOpen(SDK)_快速开发指导
[2] Quectel_LTE_Standard(U)系列_QuecOpen(SDK)_GPIO_API_参考手册
[3] Quectel_LTE_Standard(U)系列_QuecOpen(SDK)_SPI_开发指导
[4] Quectel_ECx00U&EGx00U&EG91xU 系列_QuecOpen(SDK)_Log_抓取指导
[5] Quectel_ECx00G&EGx00G 系列_QuecOpen(SDK)_Log_抓取指导

表 5: 术语缩写

缩写	英文全称	中文全称
API	Application Programming Interface	应用程序接口
CS	Chip Select	片选
DI	Data Input	数据输入
DMA	Direct Memory Access	直接存储器访问
ECC	Error Correcting Code	错误检查和纠正
EVB	Evaluation Board	评估板
FIFO	First in First Out	先进先出
GPIO	General-Purpose Input/Output	通用型输入/输出
ID	Identifier	标识符
IoT	Internet of Things	物联网
JEDEC	Joint Electron Device Engineering Council	电子器件工程联合委员会

LTE	Long-Term Evolution	长期演进
PC	Personal Computer	个人计算机
RTOS	Real-Time Operating System	实时操作系统
SD	Secure Digital Card	安全数字卡
SDK	Software Development Kit	软件开发工具包
SPI	Serial Peripheral Interface	串行外设接口
USB	Universal Serial Bus	通用串行总线